TALLINNA TEHNOLOOGIAKOLLEDŽ

Noorem tarkvaraarendaja eriala

FUNKTSIOONIDE UURIMISE VEEBIRAKENDUSE TESTIMINE

Testimisaruanne

Õpilane: Martin Rossakov, M-TARpv24

Testitava töö autor: Artjom Põldsaar, M-TARpv24

Juhendaja: Marina Oleinik

Tallinn 2026

SISUKORD

SISSEJUHATUS *4*

1 TESTI KAVANDAMINE *5*

1.1 TESTIMISE EESMÄRK JA ULATUS **5**

1.2 TESTKESKKOND **6**

1.3 TESTIMISE MEETODID **6**

1.4 TESTIDE LÄBIMISE KRITEERIUMID **7**

2 PAIGALDAMINE JA KÄIVITAMINE *8*

3 TESTJUHTUMID JA TULEMUSED *9*

3.1 ANDMEBAAS JA CRUD **9**

3.2 VALIDEERIMINE JA BACKEND **11**

3.3 KASUTAJALIIDES JA GRAAFIK **11**

3.4 MATEMAATILINE TÄPSUS **13**

3.5 TÄIENDAVATE FUNKTSIOONIDE TESTIMINE **14**

4 KOODIÜLEVAATUS *16*

4.1 ANDMEBAASIÜHENDUS JA DBCONTEXT **16**

4.2 MIGRATSIOONIDE OLEMASOLU **17**

4.3 EESTIKEELSED NIMETUSED **17**

4.4 ASÜNKROONSETE MEETODITE KASUTAMINE **18**

4.5 SERVERIPOOLNE VALIDEERIMINE **19**

4.6 KOODI LOETAVUS JA HOOLDATAVUS **20**

5 VEARAPORTID *22*

5.1 V-01. FUNKTSIOONI COS(X) ANALÜÜSITULEMUSED ON OSALISELT VALED **22**

5.2 V-02. FUNKTSIOONI (F(X)=\FRAC{1}{X^2-4}) ANALÜÜSI TULEMUSED ON OSALISELT VALED **23**

6 AUTOMAATTESTID *25*

7 HINNANG JA SOOVITUSED 26

KOKKUVÕTE 28

KASUTATUD ALLIKAD 29

LISAD 30

SISSEJUHATUS

Käesoleva töö eesmärk on testida veebirakendust FunctionExplorer, mille autor on Artjom Põldsaar. Rakenduse lähtekood on kättesaadav GitHubi repositooriumis: https://github.com/GummyisHear/FunctionExplorer. Testimise käigus loodud automaattestid asuvad testija repositooriumis: https://github.com/painkiller102k/FunctionExplorer.

Testimise eesmärk on hinnata rakenduse funktsionaalsust, kasutajaliidese kasutatavust ja matemaatiliste arvutuste korrektsust. Testimise käigus kontrollitakse funktsioonide lisamist, kuvamist, muutmist ja kustutamist, graafikute kuvamist ning funktsioonide matemaatiliste omaduste arvutamist. Samuti kontrollitakse vigaste sisendite käsitlemist ja rakenduse käivitamist.

Testimise ulatus hõlmab käsitsi funktsionaalset testimist, kasutajaliidese kontrollimist, matemaatiliste tulemuste võrdlemist käsitsi arvutatud väärtustega, lähtekoodi ülevaatust ning automaatseid ühikteste. Põhjalikku koormus- ja turvatestimist ega kõigi võimalike sisendite ammendavat kontrollimist ei tehta.

Aruanne koosneb seitsmest peatükist. Esimeses peatükis kirjeldatakse testimise kavandamist ja testkeskkonda. Teises peatükis käsitletakse rakenduse paigaldamist ja käivitamist. Kolmandas peatükis esitatakse testjuhtumid ja nende tulemused. Neljandas peatükis analüüsitakse lähtekoodi, viiendas kirjeldatakse leitud vigu, kuuendas esitatakse automaattestid ning seitsmendas antakse rakendusele hinnang ja esitatakse parandusettepanekud. Aruande lõpus on kokkuvõte, kasutatud allikad ja lisad.

Testitud lähtekoodi commit: 30e37e3cead8e92086ab4b7fbefbd77917a4d4f6.

1 TESTI KAVANDAMINE

Testimise kavandamisel määratakse testimise eesmärgid, ulatus, kasutatavad meetodid ja testkeskkond. Samuti määratakse testide läbimise kriteeriumid ning tulemuste dokumenteerimise põhimõtted. Testimise planeerimise eesmärk on tagada rakenduse süstemaatiline kontrollimine ja võimalike vigade tuvastamine.

1.1 TESTIMISE EESMÄRK JA ULATUS

Testimise eesmärk on kontrollida veebirakenduse FunctionExplorer funktsionaalsust ja matemaatiliste arvutuste korrektsust. Rakendus võimaldab sisestada matemaatilisi funktsioone, kuvada nende graafikuid ning arvutada funktsioonide määramispiirkonda, nullkohti, tuletisi, kriitilisi punkte, kasvamis- ja kahanemisvahemikke ning ekstreemumeid.

Testimise käigus kontrollitakse funktsioonide lisamist, kuvamist, muutmist ja kustutamist, graafikute kuvamist ning vigaste sisendite käsitlemist.

Rakendus on loodud C# programmeerimiskeele ja ASP.NET Core MVC raamistiku abil ning kasutab .NET 10 platvormi. Andmete salvestamiseks kasutatakse SQLite andmebaasi ja Entity Framework Core'i. Matemaatiliste avaldiste töötlemiseks kasutatakse MathNet.Symbolics teeki, mis võimaldab matemaatilisi avaldisi sümboolselt töödelda. (Math.NET, 2026)

1.2 TESTKESKKOND

Testimine toimub Windows 11 operatsioonisüsteemiga arvutis. Rakenduse käivitamiseks ja lähtekoodi haldamiseks kasutatakse Visual Studio Insidersi. Veebirakendust testitakse FireFoxi brauseris.

Tabel 1. Testkeskkond

Komponent	Kasutatav keskkond
Operatsioonisüsteem	Windows 11
Arenduskeskkond	Visual Studio Insiders
.NET SDK versioon	10.0.100
Veebibrauser	FireFox
Andmebaas	SQLite
Rakendusraamistik	ASP.NET Core MVC
Matemaatiliste avaldiste teek	MathNet.Symbolics

1.3 TESTIMISE MEETODID

Testimisel kasutatakse järgmisi meetodeid.

Funktsionaalne ehk musta kasti testimine. Kontrollitakse funktsioonide lisamist, kuvamist, muutmist ja kustutamist ning graafikute kuvamist, hinnates tegelike tulemuste vastavust oodatud tulemustele.

Piirväärtuste analüüs ja ekvivalentsklassid. Testitakse tühje, kehtivaid ja vigaseid valemeid ning funktsioone, mille määramispiirkond on piiratud.

Koodiülevaatus. Analüüsitakse lähtekoodi ülesehitust, sisendite valideerimist, andmebaasiga suhtlemist ja asünkroonsete meetodite kasutamist.

Ühiktestimine. xUnit-i abil kontrollitakse matemaatiliste arvutuste õigsust. Testid kontrollivad funktsiooni väärtuse ja tuletise arvutamist teadaolevate kontrollväärtuste põhjal.

1.4 TESTIDE LÄBIMISE KRITEERIUMID

Testjuhtumite tulemuste hindamiseks kasutatakse kolme staatust.

Tabel 2. Testjuhtumite staatuste tähendus

Staatus	Tähendus
OK	Tegelik tulemus vastab oodatud tulemusele.
VIGA	Tegelik tulemus ei vasta oodatud tulemusele.
OSALINE	Funktsioon töötab osaliselt või tulemus on puudulik.

Matemaatiliste testide puhul võrreldakse rakenduse tulemusi käsitsi arvutatud või matemaatiliste reeglite abil kontrollitud väärtustega. Avastatud vead dokumenteeritakse viiendas peatükis „Vearaportid“, lisades vea taasesitamise sammud, oodatud ja tegeliku tulemuse ning vea tõsiduse hinnangu.

2 PAIGALDAMINE JA KÄIVITAMINE

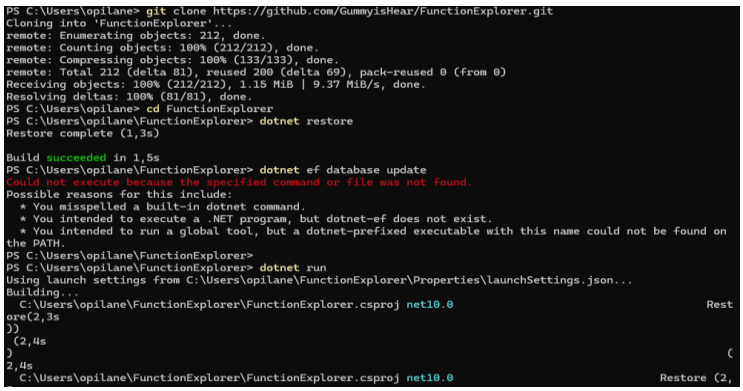
Rakenduse paigaldamist ja käivitamist kontrolliti Windows 11 operatsioonisüsteemiga arvutis. Projekti sõltuvuste taastamiseks kasutati käsku dotnet restore, mis lõpetas edukalt. Seejärel käivitati rakendus käsuga dotnet run. Rakendus käivitus edukalt ning selle kasutajaliides avanes veebibrauseris Firefox kohaliku aadressi kaudu.

Andmebaasi uuendamiseks prooviti kasutada käsku dotnet ef database update, kuid käsk ei töötanud edukalt. Seetõttu ei olnud võimalik selle abil andmebaasi migratsioone rakendada. Testimise käigus selgus, et README-failis esitatud paigaldusjuhised ei töötanud minu testkeskkonnas täielikult. Probleemi täpse põhjuse väljaselgitamiseks tuleb kontrollida veateadet, Entity Framework Core'i tööriista olemasolu ja projekti migratsioonide seadistust.

Rakenduse paigaldamiseks ja käivitamiseks kasutati järgmisi samme:

1. git clone https://github.com/GummyisHear/FunctionExplorer.git – projekti repositooriumi allalaadimine.
2. cd FunctionExplorer – projekti kataloogi sisenemine.
3. dotnet restore – projekti sõltuvuste taastamine.
4. dotnet ef database update – andmebaasi migratsioonide rakendamine. Käsk ei töötanud testkeskkonnas edukalt.
5. dotnet run – rakenduse käivitamine.

Rakenduse edukas käivitamine kinnitas, et veebirakendus on kohalikus keskkonnas kasutatav. Andmebaasi migratsioonide rakendamise probleem vajab täiendavat kontrollimist ning selle täpne põhjus tuleb võimaluse korral dokumenteerida vearaportis.



Joonis 1 "FunctionExploreri töötav veebirakendus"

3 TESTJUHTUMID JA TULEMUSED

Rakenduse FunctionExplorer testimiseks koostati testjuhtumid, mille abil kontrolliti andmebaasi ja CRUD-toimingute toimimist, sisendandmete valideerimist, kasutajaliidest, graafikute kuvamist ning matemaatiliste arvutuste täpsust. Testimine viidi läbi käsitsi, kasutades erinevaid matemaatilisi funktsioone ja vigaseid sisendeid.

Iga testjuhtumi puhul määrati eeltingimused, testimise sammud, sisendandmed, oodatav tulemus ja tegelik tulemus. Testjuhtumid märgiti läbituks (OK), ebaõnnestunuks (VIGA) või täiendavat kontrollimist vajavaks. Leitud probleemid dokumenteeritakse vearaportite peatükis.

3.1 ANDMEBAAS JA CRUD

Selles alapeatükis kontrolliti FunctionExplorer rakenduse andmebaasi ja CRUD-toimingute toimimist. Testimise käigus kontrolliti matemaatiliste funktsioonide lisamist, kuvamist, muutmist ja kustutamist. Eesmärk oli veenduda, et andmed salvestatakse korrektselt ning kasutaja tehtud muudatused kajastuvad rakenduses.

Testimisel kasutati järgmisi testjuhtumeid.

ID	Eeltingimus	Sammud	Sisend	Oodatav tulemus	Tegelik tulemus	Staatus
TJ-01	Rakendus on käivitatud ja avaleht on avatud.	1. Avada funktsiooni lisamise vorm. 2. Sisestada korrektne funktsioon. 3. Salvestada funktsioon.	x^2	Funktsioon salvestatakse andmebaasi ning kasutajale kuvatakse edukas tulemus.	Funktsioon lisati edukalt.	OK
TJ-02	Andmebaasis on vähemalt üks salvestatud funktsioon.	1. Avada funktsioonide loend. 2. Valida salvestatud funktsioon. 3. Kontrollida kuvatud andmeid.	x^2	Funktsioon kuvatakse loendis ning selle andmeid saab vaadata.	Funktsiooni andmed kuvati korrektselt.	OK
TJ-03	Andmebaasis on muudetav funktsioon.	1. Avada funktsiooni muutmise vorm. 2. Muuta funktsiooni avaldist. 3. Salvestada muudatused.	x^2 → x^3	Muudatus salvestatakse ning rakenduses kuvatakse uuendatud avaldis.	Funktsiooni muudeti edukalt.	OK
TJ-04	Andmebaasis on kustutatav funktsioon.	1. Avada funktsiooni kustutamise vaade. 2. Kinnitada kustutamine. 3. Kontrollida funktsioonide loendit.	Salvestatud funktsioon	Funktsioon kustutatakse ning seda ei kuvata enam loendis.	Funktsioon kustutati edukalt.	OK

Testimise tulemusena töötasid funktsioonide lisamine, kuvamine, muutmine ja kustutamine korrektselt. Kõik neli testjuhtumit läbisid testi edukalt.

3.2 VALIDEERIMINE JA BACKEND

Selles alapeatükis kontrolliti FunctionExplorer rakenduse sisendandmete valideerimist ja backend'i käitumist erinevate sisendite korral. Testimise eesmärk oli veenduda, et rakendus tuvastab vigased või puuduvad sisendid, kuvab kasutajale arusaadavaid veateateid ega töötle sobimatuid matemaatilisi avaldisi korrektselt vormistatud sisendina.

ID	Eeltingimus	Sammud	Sisend	Oodatav tulemus	Tegelik tulemus	Staatus
TJ-08	Rakendus on käivitatud ja funktsiooni sisestamise vorm on avatud.	1. Jätta valemi väli tühjaks. 2. Proovida vormi esitada.	Tühi sisend	Rakendus ei salvesta tühja valemit ning kuvab veateate „Valem on kohustuslik“.	Kuvati veateade „Valem on kohustuslik“.	OK
TJ-09	Funktsiooni sisestamise vorm on avatud.	1. Sisestada vigane matemaatiline avaldis. 2. Esitada vorm.	x^2 + +	Rakendus tuvastab vigase avaldise ja kuvab kasutajale arusaadava veateate.	Kuvati veateade „Valemit ei õnnestunud tõlgendada. Näide: x^3 - 3*x“.	OK
TJ-10	Funktsiooni sisestamise vorm on avatud.	1. Sisestada lubamatuid märke sisaldav tekst. 2. Esitada vorm.	abc123!!!	Rakendus tuvastab lubamatud märgid ja kuvab vastava veateate.	Kuvati veateade „Valem sisaldab lubamatuid märke“.	OK

3.3 KASUTAJALIIDES JA GRAAFIK

Selles alapeatükis kontrolliti FunctionExplorer rakenduse kasutajaliidese toimimist, veateadete kuvamist ja matemaatiliste graafikute esitamist. Testimise eesmärk oli veenduda, et rakendus töötleb vigaseid sisendeid korrektselt, kuvab matemaatilistele funktsioonidele vastavaid graafikuid ning töötab veebibrauseris õigesti.

ID	Eeltingimus	Sammud	Sisend	Oodatav tulemus	Tegelik tulemus	Staatus
TJ-13	Rakendus on avatud.	1. Sisestada vigane valem. 2. Esitada vorm. 3. Kontrollida veateadet.	x^2 + +	Rakendus kuvab arusaadava veateate, mis aitab kasutajal sisendi parandada.	Kuvati veateade vigase valemi kohta.	OK
TJ-15	Rakendus võimaldab funktsioone analüüsida.	1. Sisestada funktsioon x^2. 2. Käivitada analüüs. 3. Vaadata graafikut.	x^2	Kuvatakse parabool, mis vastab funktsioonile (f(x)=x^2).	Graafik kuvati.	OK
TJ-17	Rakendus võimaldab analüüsida murdfunktsioone.	1. Sisestada funktsioon 1/x. 2. Käivitada analüüs. 3. Kontrollida graafikut nullpunkti lähedal.	1/x	Graafikul kuvatakse kaks haru ning funktsiooni ei kuvata punktis (x=0).	Graafikul kuvati funktsiooni kaks haru ning punktis \(x=0\) katkeb funktsiooni graafik.	OK
TJ-18	Rakendus võimaldab analüüsida trigonomeetrilisi funktsioone.	1. Sisestada funktsioon sin(x). 2. Käivitada analüüs. 3. Kontrollida graafiku kuju.	sin(x)	Kuvatakse perioodiline siinusfunktsiooni graafik.	Siinusfunktsiooni perioodiline graafik kuvati korrektselt.	OK
TJ-19	Rakendus võimaldab analüüsida erinevaid funktsioone.	1. Analüüsida funktsiooni x^2. 2. Sisestada uus funktsioon x^3. 3. Käivitada analüüs uuesti.	Esmalt x^2, seejärel x^3	Graafik ja analüüsi tulemused uuenevad vastavalt uuele funktsioonile.	Pärast uue funktsiooni sisestamist uuendati graafik ja analüüsi tulemused vastavalt uuele funktsioonile.	OK
TJ-20	Rakendus on veebibrauseris avatud.	1. Avada rakendus. 2. Muuta brauseriakna laiust. 3. Kontrollida vormide ja graafiku kuvamist kitsamas aknas.	Erinevad akna laiused	Kasutajaliides jääb loetavaks ning põhifunktsioonid on kasutatavad.	Kasutajaliides ja graafik jäid brauseriakna laiuse muutmisel loetavaks ning põhifunktsioonid olid kasutatavad.	OK

TJ-21	Rakendus on avatud.	1. Avada rakendus Firefoxi brauseris. 2. Kontrollida lehe laadimist, vorme ja graafikut.	Firefox	Rakendus kuvatakse korrektselt ning selle põhifunktsioonid on kasutatavad.	Rakendus avanes ja töötas Firefoxis.	OK

3.4 MATEMAATILINE TÄPSUS

Selles alapeatükis kontrolliti FunctionExplorer rakenduse matemaatiliste arvutuste täpsust. Testimise käigus võrreldi rakenduse arvutatud tulemusi käsitsi arvutatud matemaatiliste tulemustega. Kontrolliti funktsioonide määramispiirkondi, tuletisi, nullkohti, kriitilisi punkte, kasvamis- ja kahanemisvahemikke ning ekstreemume.

ID	Eeltingimus	Sammud	Sisend	Oodatav tulemus	Tegelik tulemus	Staatus
TJ-22	Rakendus võimaldab matemaatilisi funktsioone analüüsida.	1. Sisestada funktsioon. 2. Käivitada analüüs. 3. Võrrelda tulemusi käsitsi arvutatud tulemustega.	`x^2`	Määramispiirkond on (\mathbb{R}), nullkoht on (x=0), tuletis on (f'(x)=2x) ning miinimum asub punktis ((0,0)).	Rakendus kuvas määramispiirkonna, nullkoha, tuletise ja miinimumi korrektselt.	OK
TJ-23	Rakendus võimaldab kuupfunktsioone analüüsida.	1. Sisestada funktsioon. 2. Käivitada analüüs. 3. Kontrollida nullkohti, tuletist ja ekstreemume.	`x^3-3*x`	Nullkohad on (x=-\sqrt{3}), (x=0) ja (x=\sqrt{3}). Tuletis on (f'(x)=3x^2-3). Lokaalne maksimum on punktis ((-1,2)) ja lokaalne miinimum punktis ((1,-2)).	Rakendus kuvas nullkohad, tuletise ja ekstreemumid korrektselt.	OK
TJ-24	Rakendus võimaldab murdfunktsioone analüüsida.	1. Sisestada funktsioon. 2. Käivitada analüüs. 3. Kontrollida määramispiirkonda ja tuletist.	`1/x`	Määramispiirkond on (\mathbb{R}\setminus{0}), nullkohti ei ole ning tuletis on (f'(x)=-1/x^2). Funktsioon kahaneb vahemikes ((-\infty,0)) ja ((0,\infty)).	Rakendus kuvas määramispiirkonna, tuletise ja kahanemisvahemikud korrektselt.	OK
TJ-25	Rakendus võimaldab analüüsida neljanda astme funktsioone.	1. Sisestada funktsioon. 2. Käivitada analüüs. 3. Kontrollida nullkohti,	`x^4-4*x^2`	Nullkohad on (x=-2), (x=0) ja (x=2). Tuletis on (f'(x)=4x^3-8x). Lokaalsed miinimumid asuvad punktides ((-\sqrt{2},-4)) ja	Rakendus kuvas nullkohad, tuletise ja ekstreemumid korrektselt.	OK

		tuletist ja ekstreemume.		((\sqrt{2},-4)), lokaalne maksimum asub punktis ((0,0)).		

Testimise tulemusena vastasid valitud funktsioonide määramispiirkonnad, nullkohad, tuletised ja ekstreemumid matemaatilistele arvutustele. Tulemuste võrdlemine käsitsi arvutatud väärtustega aitas hinnata rakenduse matemaatiliste arvutuste usaldusväärsust.

3.5 TÄIENDAVATE FUNKTSIOONIDE TESTIMINE

Selles alapeatükis kontrolliti FunctionExplorer rakenduse tööd erinevate matemaatiliste funktsioonidega. Testimise eesmärk oli hinnata, kas rakendus suudab analüüsida erineva kujuga funktsioone ning kuvada nende omadused korrektselt.

ID	Eeltingimus	Sammud	Sisend	Oodatav tulemus	Tegelik tulemus	Staatus
TJ-26	Rakendus võimaldab matemaatilisi funktsioone analüüsida.	1. Sisestada eksponentfunktsioon. 2. Käivitada analüüs. 3. Kontrollida tuletist ja kasvamisvahemikku.	e^x	Tuletis on (f'(x)=e^x) ning funktsioon kasvab kogu määramispiirkonnas.	Rakendus kuvas tuletise ja kasvamisvahemiku korrektselt.	OK
TJ-27	Rakendus võimaldab trigonomeetrilisi funktsioone analüüsida.	1. Sisestada koosinusfunktsioon. 2. Käivitada analüüs. 3. Kontrollida tuletist.	cos(x)	Tuletis on (f'(x)=-\sin(x)).	Tuletis kuvati korrektselt, kuid kasvamis- ja kahanemisvahemikud ning ekstreemumid ei olnud täielikult korrektsed.	VIGA
TJ-28	Rakendus võimaldab ratsionaalfunktsioone analüüsida.	1. Sisestada ratsionaalfunktsioon. 2. Käivitada analüüs. 3. Kontrollida kasvamis- ja kahanemisvahemikke.	1/(x^2-4)	Funktsioon kasvab vahemikes ((-\infty,-2)) ja ((-2,0)) ning kahaneb vahemikes ((0,2)) ja ((2,\infty)).	Kasvamis- ja kahanemisvahemikud kuvati osaliselt valesti.	VIGA

Testimise tulemusena selgus, et rakendus töötles eksponentfunktsiooni korrektselt, kuid mõne trigonomeetrilise ja ratsionaalfunktsiooni analüüsimisel esines ebatäpsusi. Leitud probleemid näitavad, et kasvamis- ja kahanemisvahemike ning ekstreemumite arvutamist tuleks täiustada.

4 KOODIÜLEVAATUS

Koodiülevaatuse eesmärk oli hinnata FunctionExplorer rakenduse lähtekoodi struktuuri, andmebaasiühenduse seadistust, nimetamispraktikaid, asünkroonsete meetodite kasutamist, serveripoolset valideerimist ja koodi loetavust. Ülevaatus tehti projekti lähtekoodi põhjal.

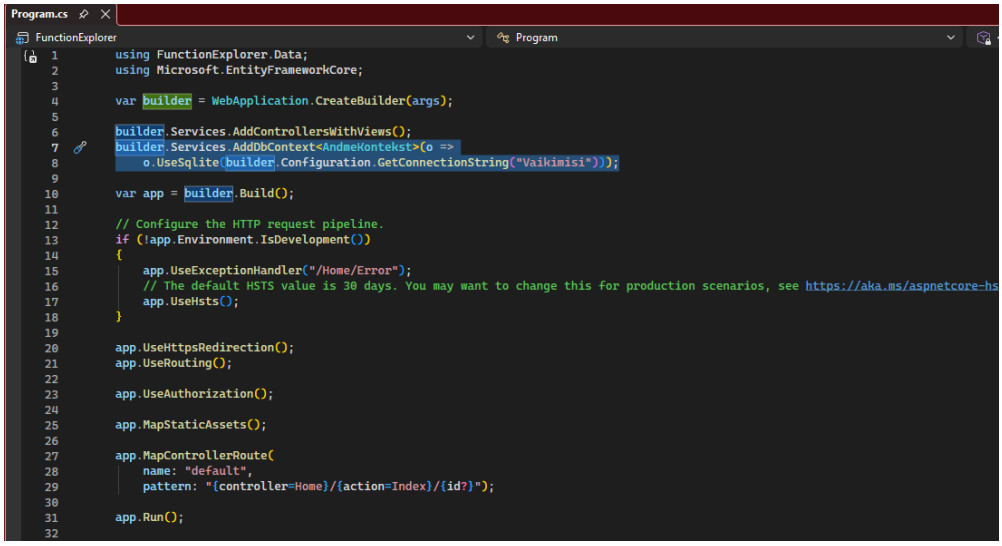
4.1 ANDMEBAASIÜHENDUS JA DBCONTEXT

Failis Program.cs on andmebaasi kontekst AndmeKontekst registreeritud sõltuvussüstimise teenusena. Ridadel 7–8 kasutatakse meetodit AddDbContext ning SQLite'i ühenduse seadistamiseks loetakse ühendusstring konfiguratsioonist võtmega Vaikimisi.

Failis appsettings.json asub ühendusstring real 3: Data Source=funktsioonid.db. Failis Data/DataContext.cs on määratletud klass AndmeKontekst, mis pärineb klassist DbContext. See näitab, et rakendus kasutab andmebaasiga töötamiseks Entity Framework Core'i ja SQLite'i. Microsofti dokumentatsioon selgitab, kuidas Entity Framework Core'i rakenduses andmebaasi ühendusstringi konfigureerida. (Microsoft, 2024)

Hinnang: nõue on täidetud.

Tõendid: Program.cs, read 7–8; appsettings.json, rida 3; Data/DataContext.cs, read 6–10.



Joonis 2"Andmebaasi konteksti registreerimine failis Program.cs"

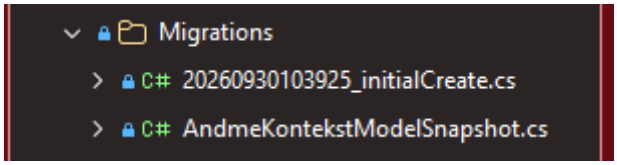
4.2 MIGRATSIOONIDE OLEMASOLU

Projektis on olemas kaust Migrations, milles asuvad failid 20260930103925_initialCreate.cs, 20260930103925_initialCreate.Designer.cs ja AndmeKontekstModelSnapshot.cs. Need failid näitavad, et projektis kasutatakse Entity Framework Core'i migratsioonimehhanismi andmebaasi struktuuri haldamiseks.

Migratsioonifailide olemasolu kinnitab, et andmebaasi struktuuri muudatusi on võimalik hallata migratsioonide abil. Koodiülevaatuse käigus ei kontrollitud eraldi, kas kõik migratsioonid rakenduvad veatult täiesti puhtas andmebaasis.

Hinnang: migratsioonifailid on repositooriumis olemas, kuid nende rakendumist tuleb vajaduse korral eraldi testida.

Tõendid: projekti kaust Migrations ja selles olevad nimetatud failid.



Joonis 3"Entity Framework Core'i migratsioonifailid projektis"

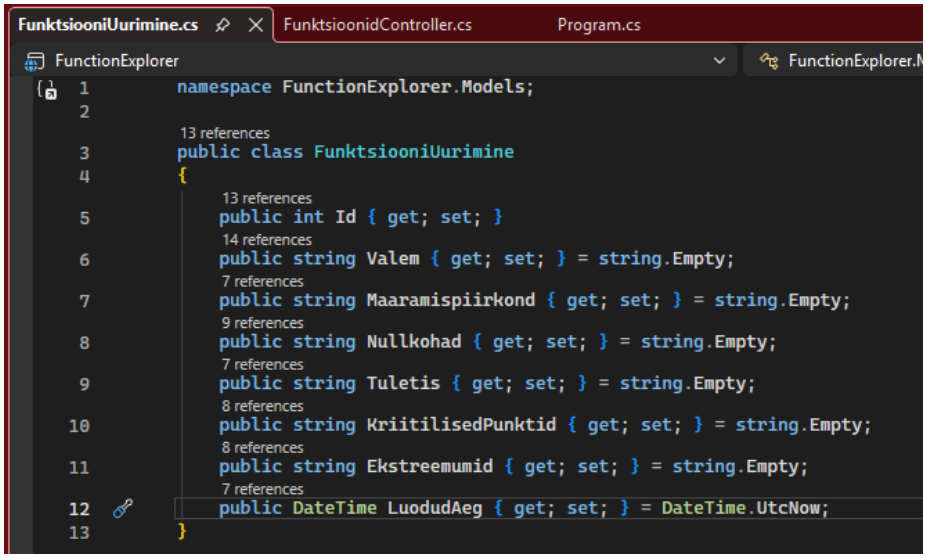
4.3 EESTIKEELSED NIMETUSED

Mudelite ja andmebaasikonteksti nimetamisel kasutatakse eestikeelseid termineid. Näiteks on olemas klassid FunktsiooniUurimine, FunktsiooniVorm ja AndmeKontekst. Samuti on omaduste nimed eestikeelsed: Valem, Maaramispiirkond, Nullkohad, Tuletis, KriitilisedPunktid, Ekstreemumid ja LuodudAeg.

Samas kasutatakse kontrolleris ka ingliskeelseid tegevusmeetodite nimetusi, näiteks Index, Details, Create, Edit ja Delete. Seega on mudelite ja nende väljade nimetamine eestikeelne, kuid kogu projektis ei ole nimetamispraktika täielikult ühtne.

Hinnang: mudelite ja väljade nimetused vastavad nõudele, kuid kogu koodis võiks nimetamispraktika olla ühtlasem.

Tõendid: Models/FunktsiooniUurimine.cs, read 3–12; Models/FunktsiooniVorm.cs, read 5–13; Controllers/FunktsioonidController.cs, read 16–20 ja 32–35.



Joonis 4"Eestikeelsete nimetustega mudel FunktsiooniUurimine"

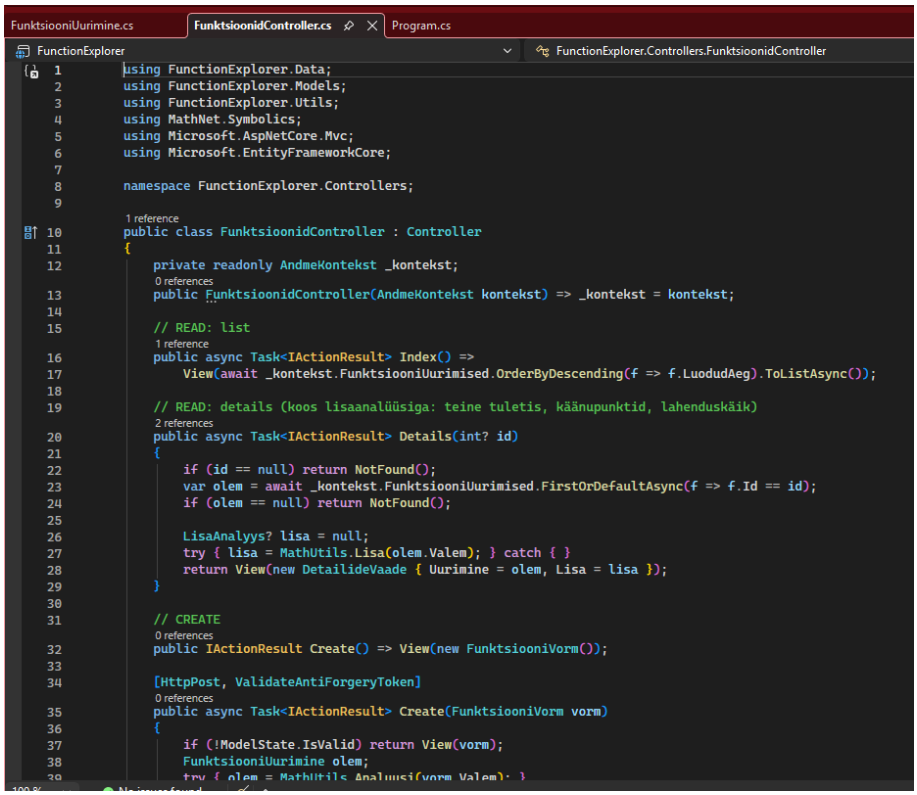
4.4 ASÜNKROONSETE MEETODITE KASUTAMINE

Andmebaasipäringutes ja andmete salvestamisel kasutatakse asünkroonseid meetodeid. Näiteks tagastab Index-meetod funktsioonide loendi meetodiga ToListAsync(). Uue kirje lisamisel ja olemasoleva kirje muutmisel kasutatakse meetodit SaveChangesAsync(). Kirjete otsimiseks kasutatakse ka meetodeid FirstOrDefaultAsync() ja FindAsync().

Asünkroonne lähenemine aitab vältida päringute ja salvestamistoimingute ajal päringut käivitava lõime tarbetut blokeerimist.

Hinnang: nõue on täidetud, sest kontrolleris kasutatakse mitut asünkroonset andmebaasimeetodit.

Tõendid: Controllers/FunktsioonidController.cs, read 16–17, 23, 45–46, 54 ja 74.



Joonis 5"Asünkroonsete andmebaasimeetodite kasutamine"

4.5 SERVERIPOOLNE VALIDEERIMINE

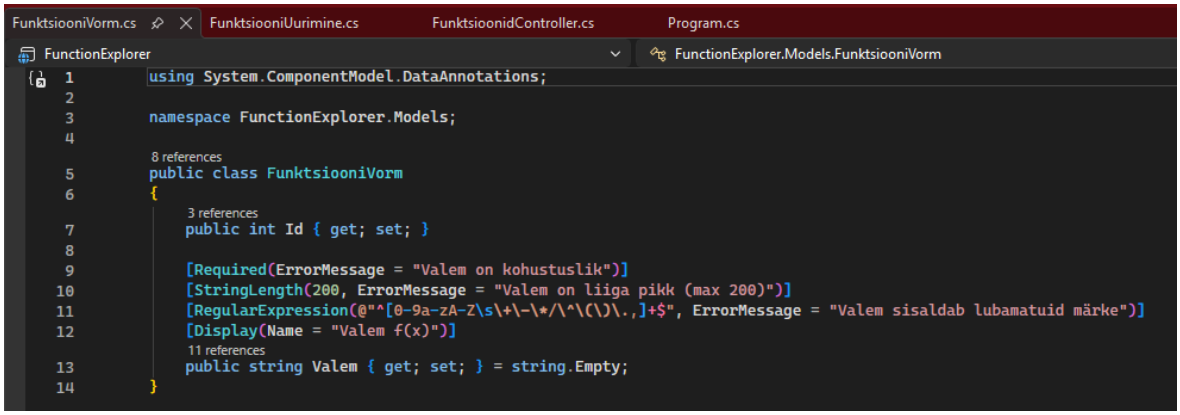
Serveripoolne valideerimine on rakendatud mudelis FunktsiooniVorm. Omadusele Valem on lisatud atribuudid [Required], [StringLength(200)] ja [RegularExpression(...)]. Nende abil kontrollitakse, et valem oleks sisestatud, ei ületaks 200 märki ega sisaldaks lubamatuid märke.

Kontrolleris kontrollitakse ModelState.IsValid väärtust nii uue kirje loomisel kui ka olemasoleva kirje muutmisel. Kui valem ei vasta nõuetele, tagastatakse vorm koos valideerimisandmetega. Valemi analüüsimisel tekkivate vigade korral lisatakse kasutajale veateade meetodiga ModelState.AddModelError().

Lisaks kasutatakse POST-päringutega tegevustes atribuuti [ValidateAntiForgeryToken], mis aitab kaitsta vormide kaudu tehtavaid päringuid võltsitud päringute eest.

Hinnang: serveripoolne valideerimine on olemas nii mudeli atribuutide kui ka kontrolleri tasemel.

Tõendid: Models/FunktsiooniVorm.cs, read 9–13; Controllers/FunktsioonidController.cs, read 34–44, 58–70 ja 79–80.



Joonis 6 "Valemi serveripoolne valideerimine mudelis FunktsiooniVorm"

4.6 KOODI LOETAVUS JA HOOLDATAVUS

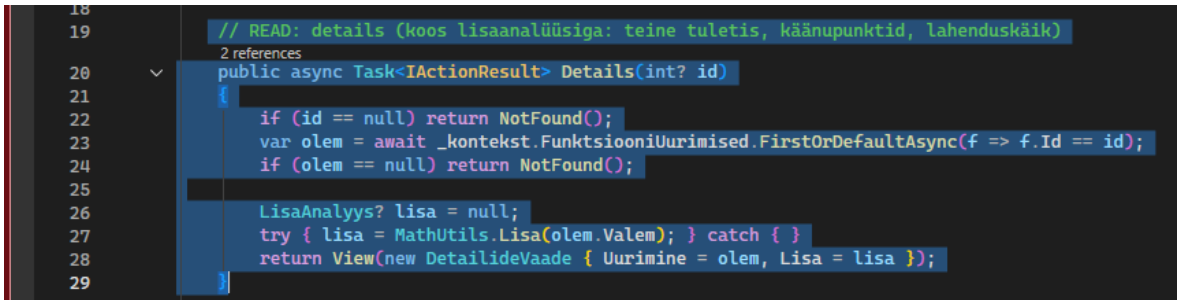
Kood on jagatud loogilisteks osadeks: kontrollerid, mudelid, andmekontekst ja matemaatilised abimeetodid asuvad eraldi kaustades. Klasside ja omaduste nimed on enamasti kirjeldavad ning kontrolleris on kasutatud kommentaare, mis tähistavad erinevaid toiminguid, näiteks andmete lugemist ja kustutamist.

Samas esineb kohti, mida saab parandada. Meetodis Details on tühi catch-plokk, mis peidab lisaanalüüsi käigus tekkinud vea ega anna selle kohta infot. See võib muuta vea põhjuse leidmise keerulisemaks. Samuti on mõnes kohas mitu omistamist kirjutatud ühele reale, mistõttu võiks koodi vormindust parandada.

Üldiselt on koodi struktuur arusaadav, kuid veakäsitlus võiks olla läbipaistvam ning vormindus ühtlasem.

Hinnang: kood on üldjoontes loetav ja loogiliselt struktureeritud, kuid hooldatavust saab parandada.

Tõendid: Controllers/FunktsioonidController.cs, read 19–28 ja 71–74.



```
19      // READ: details (koos lisaanalüüsiga: teine tuletis, käänupunktid, lahenduskäik)
        2 references
20      public async Task<IActionResult> Details(int? id)
21      {
22          if (id == null) return NotFound();
23          var olem = await _kontekst.FunktsiooniUurimised.FirstOrDefaultAsync(f => f.Id == id);
24          if (olem == null) return NotFound();
25
26          LisaAnalyys? lisa = null;
27          try { lisa = MathUtils.Lisa(olem.Valem); } catch { }
28          return View(new DetailideVaade { Uurimine = olem, Lisa = lisa });
29      }
```

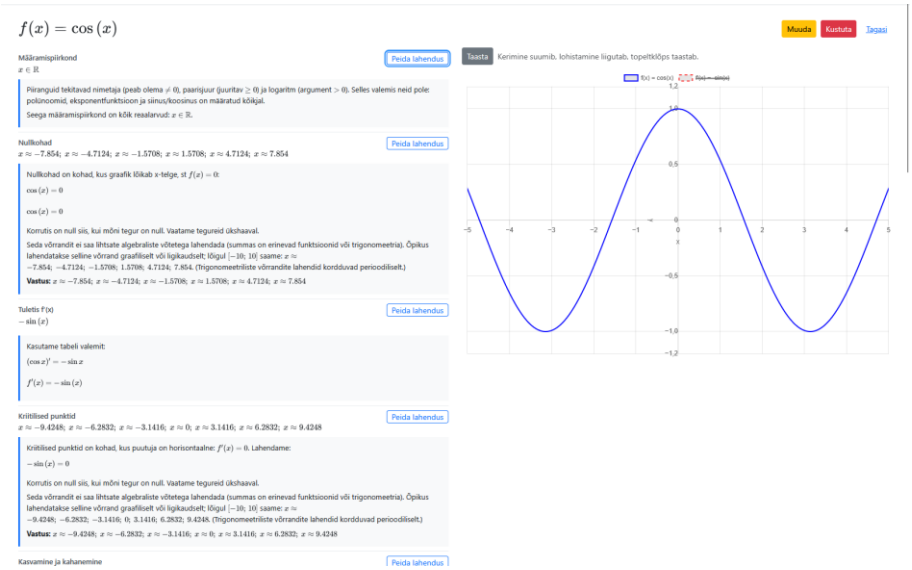
Joonis 7"Vea käsitlemine Details-meetodis"

5 VEARAPORTID

Testimise käigus leiti kaks viga, mis on seotud matemaatiliste funktsioonide analüüsiga. Mõlemal juhul kuvab rakendus osa analüüsitulemustest valesti. Kuna matemaatiline analüüs on rakenduse põhifunktsioon, võivad sellised vead mõjutada kasutaja usaldust saadud tulemuste vastu.

5.1 V-01. FUNKTSIOONI COS(X) ANALÜÜSITULEMUSED ON OSALISELT VALED

Väli	Kirjeldus
Vea ID	V-01
Seotud TJ	TJ-27
Kirjeldus	Funktsiooni cos(x) analüüsimisel ei kuvata kõiki monotoonsuse intervalle ja ekstreemumeid korrektselt. Tuletis kuvatakse õigesti kujul -sin(x), kuid osa ülejäänud analüüsitulemustest on ebatäielikud või valed.
Taasesitamise sammud	1. Ava FunctionExplorer rakendus. 2. Ava funktsiooni lisamise või analüüsimise vorm. 3. Sisesta valem cos(x). 4. Käivita analüüs. 5. Kontrolli tuletist, monotoonsuse intervalle, ekstreemumeid ja kumerusintervalle.
Oodatav / tegelik	**Oodatav:** rakendus kuvab funktsiooni cos(x) matemaatiliselt õiged monotoonsuse intervallid, ekstreemumid ja kumerusintervallid. **Tegelik:** tuletis -sin(x) kuvatakse õigesti, kuid osa monotoonsuse intervalle on valed ning ekstreemumite ja kumeruse tulemused on ebatäielikud või valed.
Tõsidus	Kõrge
Tõend	Lisa 2, joonis 7 – funktsiooni cos(x) analüüsitulemuste ekraanipilt.

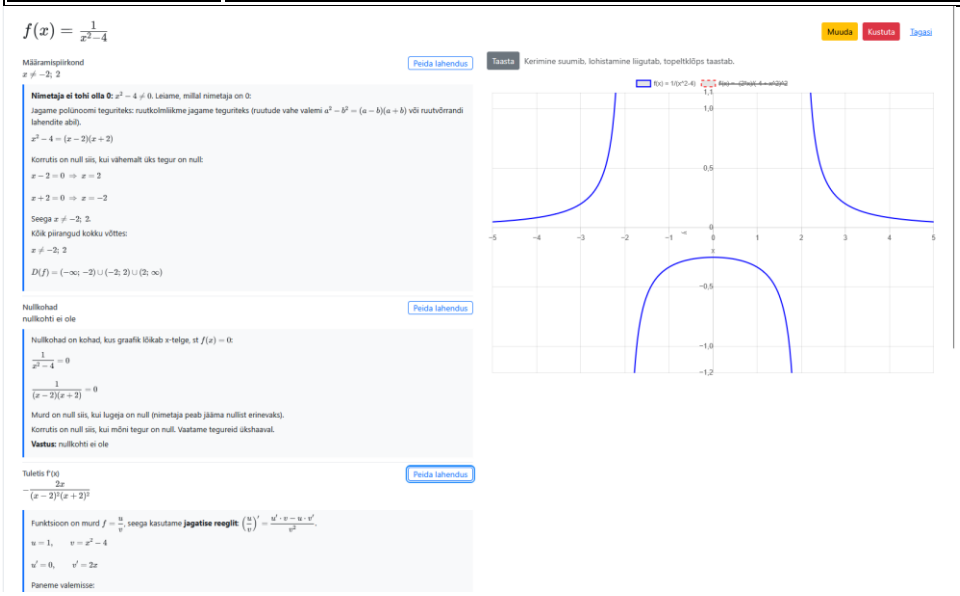


Joonis 8"Funktsiooni cos(x) analüüsitulemused"

5.2 V-02. FUNKTSIOONI (F(X)=\FRAC{1}{X^2-4}) ANALÜÜSI TULEMUSED ON OSALISELT VALED

Väli	**Kirjeldus**
Vea ID	V-02
Seotud TJ	TJ-28
Kirjeldus	Funktsiooni (f(x)=\frac{1}{x^2-4}) analüüsimisel ei kuvata kõiki tulemusi õigesti. Funktsiooni tuletis on (f'(x)=\frac{-2x}{(x^2-4)^2}). Funktsioon kasvab vahemikes ((-\infty;-2)) ja ((-2;0)) ning kahaneb vahemikes ((0;2)) ja ((2;+\infty)). Funktsiooni maksimum on punktis ((0;-\frac14)). Funktsioon ei ole määratud punktides (x=-2) ja (x=2).
Taasesitamise sammud	1. Ava FunctionExplorer rakendus. 2. Ava funktsiooni lisamise või analüüsimise vorm. 3. Sisesta valem 1/(x^2-4). 4. Käivita analüüs. 5. Võrdle kuvatud tuletist, monotoonsuse intervalle ja ekstreemumipunkte matemaatiliselt õigete tulemustega.

Oodatav / tegelik	**Oodatav:** Rakendus kuvab õige tuletise, kasvamisintervallid (-∞, -2) ja (-2, 0), kahanemisintervallid (0, 2) ja (2, ∞) ning maksimumi punktis (0, -1/4). Punktides x = -2 ja x = 2 ei ole funktsioon määratud. **Tegelik:** Rakenduse kuvatud analüüsitulemused ei vasta täielikult oodatud tulemustele. Täpne erinevus tuleb kirjeldada rakenduse tegeliku väljundi ja lisatud ekraanipildi põhjal.
Tõsidus	Kõrge
Tõend	Lisa 2, joonis 8 – funktsiooni 1/(x^2-4) analüüsitulemuste ekraanipilt.



Joonis 9"Funktsiooni 1/(x² − 4) analüüsitulemused"

6 AUTOMAATTESTID

Rakenduse matemaatilise osa kontrollimiseks kasutati eraldi xUnit-testprojekti FunctionExplorer.Tests. xUnit on testimisraamistik, mida kasutatakse automaattestide kirjutamiseks ja käivitamiseks. Testid kontrollivad klassi MathUtils meetodite Arvuta ja Tuletis tööd. Testides ei kasutata andmebaasi ega mock-objekte. (xUnit, 2026)

Esimene ühiktest kontrollib funktsiooni (f(x)=x^2) väärtuse arvutamist punktis (x=3). Oodatav tulemus on 9, sest (3^2=9).

Teine ühiktest kontrollib sama funktsiooni tuletise arvutamist punktis (x=3). Funktsiooni (f(x)=x^2) tuletis on (f'(x)=2x), seega oodatav tulemus on 6.

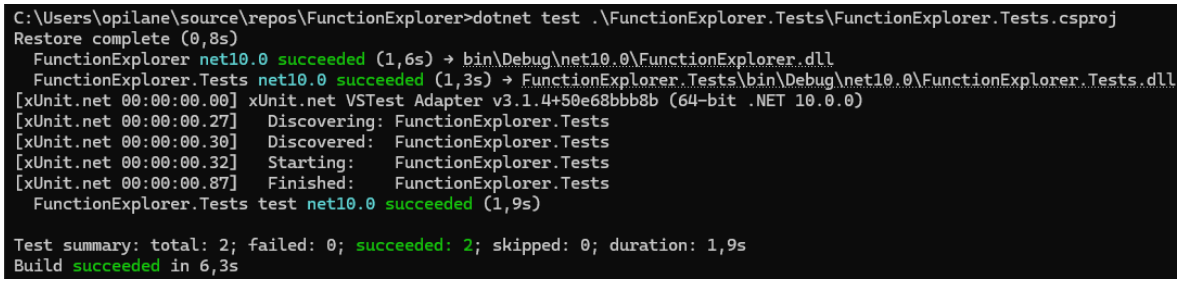
Testides kasutatakse Assert.NotNull kontrolli, et tulemus oleks olemas, ja Assert.Equal kontrolli, et arvutatud väärtus vastaks oodatud tulemusele kuue kümnendkoha täpsusega.

Testid käivitati käsuga dotnet test .\FunctionExplorer.Tests\FunctionExplorer.Tests.csproj. Mõlemad testid läbisid kontrolli edukalt: kokku käivitati kaks testi, millest kaks õnnestusid ja ükski ei ebaõnnestunud ega jäänud vahele.

Testide tulemuste kuvatõmmis on esitatud lisas 3.

Testide tulemused:

- Arvuta_XRuutKuiXOnKolm_TagastabUheksa – kontrollib, et (f(3)=9).
- Tuletis_XRuutKuiXOnKolm_TagastabKuus – kontrollib, et (f'(3)=6).



Joonis 10" xUnit-ühiktestide käivitamise tulemus. Mõlemad testid läbisid edukalt: kokku käivitati 2 testi, millest 2 õnnestusid ja 0 ebaõnnestusid."

7 HINNANG JA SOOVITUSED

1. Parandada matemaatilise analüüsi algoritme. Kontrollida trigonomeetriliste funktsioonide monotoonsuse intervalle, ekstreemumeid ja kumerusintervalle. Esmajärjekorras tuleb parandada funktsiooni cos(x) analüüs. (V-01, TJ-19, TJ-27)
2. Parandada vigade käsitlemist. Vältida tühje catch-plokke ning tagada, et ootamatud arvutusvead ei jääks märkamata. Vajaduse korral kuvada kasutajale selge veateade ja salvestada tehniline info logisse.
3. Laiendada kasutajaliidese testimist. Kontrollida rakenduse kuvamist erineva laiusega ekraanidel, klaviatuuriga kasutamist ning toimimist teises veebibrauseris. Praegu on rakendust testitud Firefoxis.
4. Muuta kasutajaliides mugavamaks ja lihtsamaks, vähendades liigset teksti ning muutes olulised funktsioonid paremini leitavaks.

Kriteerium	Maks. p	Hinnang	Põhjendus (viide TJ / V)
EF Core ja andmebaas	30	30	CRUD-toimingud töötasid edukalt: funktsiooni lisamine, kuvamine, muutmine ja kustutamine õnnestusid. Andmebaasi kontekst ja migratsioonid on projektis olemas. (TJ-01–TJ-04)
Backend ja valideerimine	25	25	Tühja valemi, vigase süntaksi ja lubamatute märkide korral kuvati veateated. Serveripoolne valideerimine toimub ModelState abil. (TJ-08–TJ-10)
Graafik ja kasutajaliides	30	20	Graafik kuvati ning rakendus töötas Firefoxi brauseris. Mõne funktsiooni analüüsitulemused olid siiski ebatäielikud. (TJ-15, TJ-18–TJ-21, V-01)
Matemaatiline täpsus	15	10	Mitme funktsiooni arvutustulemused olid õiged, kuid trigonomeetriliste funktsioonide analüüsis esines ebatäpsusi. Parandamist vajavad funktsiooni cos(x) monotoonsuse intervallid, ekstreemumid ja kumerusintervallid. (TJ-19, TJ-27, V-01)
Boonusülesanne	+15	+10	Rakendus kasutab MathNet.Symbolics teeki matemaatiliste avaldiste töötlemiseks ning pakub täiendavat funktsionaalsust graafiku punktide arvutamiseks. (FunktsioonidController.cs, meetod Punktid)
KOKKU	**100(+15)**	**85 (+10)**	Rakenduse põhifunktsioonid töötavad edukalt, kuid matemaatilise analüüsi täpsust ja kasutajaliidest on võimalik veel parandada.

KOKKUVÕTE

Testimise eesmärk oli hinnata FunctionExploreri veebirakenduse töökindlust, andmebaasitoiminguid, sisendi valideerimist, kasutajaliidest ja matemaatiliste arvutuste täpsust.

Testimise käigus kontrolliti funktsioonide lisamist, kuvamist, muutmist ja kustutamist ning erinevate matemaatiliste avaldiste analüüsimist. Põhilised andmebaasitoimingud ja sisendi valideerimine töötasid edukalt. Samuti käivitati automaattestid, millest mõlemad läbisid edukalt.

Testimisel ilmnes, et mõne trigonomeetrilise funktsiooni analüüsitulemused ei olnud täielikud ega piisavalt täpsed. Eelkõige vajab parandamist funktsiooni cos(x) monotoonsuse, ekstreemumite ja kumerusintervallide leidmine. Lisaks saab parandada vigade käsitlemist ja muuta kasutajaliidese lihtsamaks ning mugavamaks.

Kokkuvõttes on FunctionExplorer kasutatav rakendus, mille põhifunktsioonid töötavad, kuid matemaatilise analüüsi täpsuse ja kasutajakogemuse parandamiseks on vaja teha täiendusi. Testimine aitas tuvastada rakenduse tugevused ja puudused ning määrata edasised arendussuunad.

KASUTATUD ALLIKAD

Math.NET. (22. 7 2026. a.). *Math.NET Symbolics*. Allikas: GitHub: https://github.com/mathnet/mathnet-symbolics

Microsoft. (12. 11 2024. a.). *Connection Strings – EF Core*. Allikas: Microsoft Learn: https://learn.microsoft.com/ef/core/miscellaneous/connection-strings

xUnit. (12. 1 2026. a.). *xUnit.net*. Allikas: xUnit.net: https://xunit.net/

LISAD

UnitTest1.cs fail

```
1
2   using FunctionExplorer.Utils;
3   using MathNet.Symbolics;
4   using Xunit;
5
6   namespace FunctionExplorer.Tests;
7
8   public class MathUtilsTests
9   {
10      [Fact]
11      public void Arvuta_XRuutKuiXOnKolm_TagastabUheksa()
12      {
13          var funktsioon = SymbolicExpression.Parse("x^2");
14
15          var tulemus = MathUtils.Arvuta(funktsioon, 3);
16
17          Assert.NotNull(tulemus);
18          Assert.Equal(9, tulemus.Value, 6);
19      }
20
21      [Fact]
22      public void Tuletis_XRuutKuiXOnKolm_TagastabKuus()
23      {
24          var funktsioon = SymbolicExpression.Parse("x^2");
25          var tuletis = MathUtils.Tuletis(funktsioon);
26
27          var tulemus = MathUtils.Arvuta(tuletis, 3);
28
29          Assert.NotNull(tulemus);
30          Assert.Equal(6, tulemus.Value, 6);
31      }
32  }
```

Joonis 11 "fail UniTest1.cs"